

知枢

企业知识资产管理与智能检索平台

执行手册（重排版版）

课程设计方向：自选题（企业知识资产管理与智能检索平台）

技术栈：React + TypeScript + Rust + MySQL + Qdrant + Redis + 大模型接口

文档用途：需求分析 / 数据库设计 / 前后端设计 / Agent 设计 / 项目执行与答辩

说明：本版重点修正 ER 图排版问题。ER 图严格按照“实体—属性—联系”连线，不在图中标注主键/外键；主键、外键统一在关系模型表中说明。为提高可读性，ER 图按三个业务域拆分。

目录

1. 项目概述
2. 需求分析与业务范围
3. 功能模块设计
4. 技术路线与总体架构
5. 项目目录结构设计
6. 数据库总体设计原则
7. 中文实体与属性清单
8. 中文 ER 图设计（分图展示）
9. 联系说明表
10. 关系模型汇总表与详细表
11. Agent 体系与 RAG 流程
12. 前端详细设计（TypeScript）
13. 后端详细设计（Rust）
14. 大模型、向量库与检索配置
15. 环境变量与配置文件架构
16. 接口设计与交互流程
17. 开发执行计划
18. 测试计划
19. 部署方案与运维建议
20. 答辩展示建议

1. 项目概述

项目名称：企业知识资产管理与智能检索平台

系统名称：知枢

项目定位：面向中小型企业内部知识管理场景，统一管理制度文档、技术文档、项目资料、FAQ、培训资料与流程规范，并在此基础上提供智能摘要、标签推荐、相关文档推荐和知识问答能力。

建设价值：一是解决“资料分散、查找困难、版本混乱、知识复用不足”的问题；二是将大模型与结构化知识资产结合，形成可演示、可扩展、可写入简历的知识管理系统；三是满足课程设计对 Web 系统、数据库设计、前后端、测试与答辩展示的完整要求。

1.1 用户角色

角色名称	职责说明
系统管理员	维护用户、角色、分类、文档全局配置；审核文档；查看统计报表。
知识管理员	录入与维护文档、版本、标签与 FAQ；触发 Agent 摘要、标签、FAQ 生成。
普通员工	搜索、阅读、收藏文档；提交问题；查看问答结果与推荐资料。

2. 需求分析与业务范围

业务对象包括：用户、角色、分类、文档、标签、文档版本、阅读记录、收藏记录、FAQ 条目、用户问题、回答记录、智能处理记录。

2.1 业务痛点

编号	痛点	系统应对
P1	企业内部资料分散在多个文件夹或个人电脑中，查询效率低。	构建统一知识库，支持分类、标签与关键词检索。
P2	同一份资料多次修改，历史版本无法追踪。	建立文档版本管理与变更说明机制。
P3	员工重复提问，知识难以沉淀。	建设 FAQ 与知识问答功能，保存问答记录。
P4	资料阅读情况不可见，无法识别高价值文档。	记录阅读、收藏、热门文档、问答趋势。
P5	人工维护标签和摘要成本高。	通过多 Agent 自动生成摘要、关键词、标签和 FAQ。

2.2 范围边界

本阶段纳入范围：模拟企业文档的新增、发布、搜索、问答、智能处理与统计看板。

本阶段不纳入范围：复杂审批流程、真实文件 OCR、企业 LDAP 单点登录、多租户隔离、生产级权限审计。

3. 功能模块设计

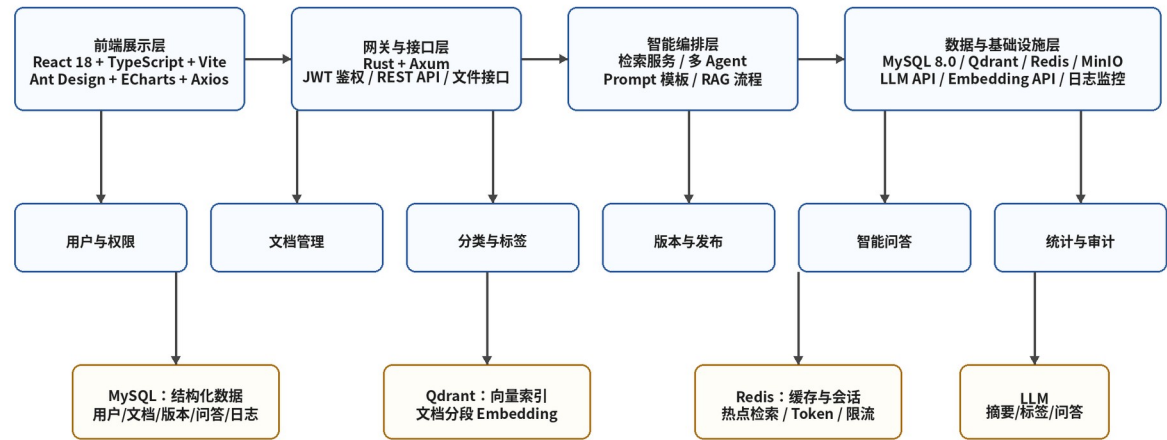
模块	子功能	说明
用户与权限	注册登录、角色管理、个人中心	区分管理员与普通员工，控制文档维护与查看权限。
文档管理	新增、编辑、删除、发布、详情查看	知识资产主表，支持正文、摘要、状态、版本号维护。
分类与标签	分类 CRUD、标签 CRUD、文档绑定标签	便于结构化管理与多维检索。
版本管理	历史版本保存、修改说明、版本切换	支持文档追溯与变更记录。
阅读与收藏	阅读记录、收藏记录、热门文档	支持用户行为分析。
智能检索	关键词检索、分类筛选、标签筛选、相关推荐	结合结构化字段与向量召回。
知识问答	提出问题、返回答案、引用文档	基于 RAG 过程组织答案。
Agent 中心	摘要、关键词、标签、FAQ、推荐、审计	多 Agent 协作完成知识增强。
统计看板	总量、趋势、热门标签、问答次数	增强系统可视化与答辩展示效果。

4. 技术路线与总体架构

前端采用 React + TypeScript + Vite 构建单页应用，后端采用 Rust + Axum 提供 REST API，MySQL 负责结构化数据持久化，Qdrant 存储文档向量，Redis 负责缓存、会话与热点数据，大模型接口采用兼容 OpenAI 协议的可替换方案。

图 4-1 系统总体技术架构

前端采用 TypeScript 技术栈，后端采用 Rust 服务化架构；知识检索由 MySQL + Qdrant + Redis 共同支撑。



架构要点：

(1) 结构化数据（用户、文档、标签、问答记录等）进入 MySQL；(2) 文档切分后的向量进入 Qdrant；(3) 热点问答和查询条件缓存进入 Redis；(4) 摘要、标签、FAQ、问答由多 Agent 调用大模型或规则模板生成；(5) 前后端完全分离，便于协作开发。

用户	用户编号、用户名、密码摘要、邮箱、部门、创建时间
分类	分类名称、分类说明
文档	文档标题、正文内容、摘要、状态、版本号、更新时间
标签	标签名称、标签说明
文档版本	版本号、历史正文、修改说明、创建时间
阅读记录	阅读时间
收藏记录	收藏时间
FAQ 条目	问题、答案、来源文档
用户问题	问题内容、提问时间
回答记录	回答内容、置信度、回答时间
智能处理记录	处理类型、输入内容、输出内容、处理时间

8. 中文 ER 图设计（分图展示）

说明：为解决上一版总图过于拥挤、关系交叉过多、部分属性不可读的问题，本版将 ER 设计拆分为三张专题图。三张图共同构成完整的逻辑 ER 设计。每张图均严格采用“实体—属性—联系”连线。

8.1 图 8-1 组织与文档主数据

图 8-1 组织与文档主数据 ER 图（Chen 表示法）

说明：矩形为实体，椭圆为属性；菱形为联系；基数标注在联系两侧。图中不标主键/外键。

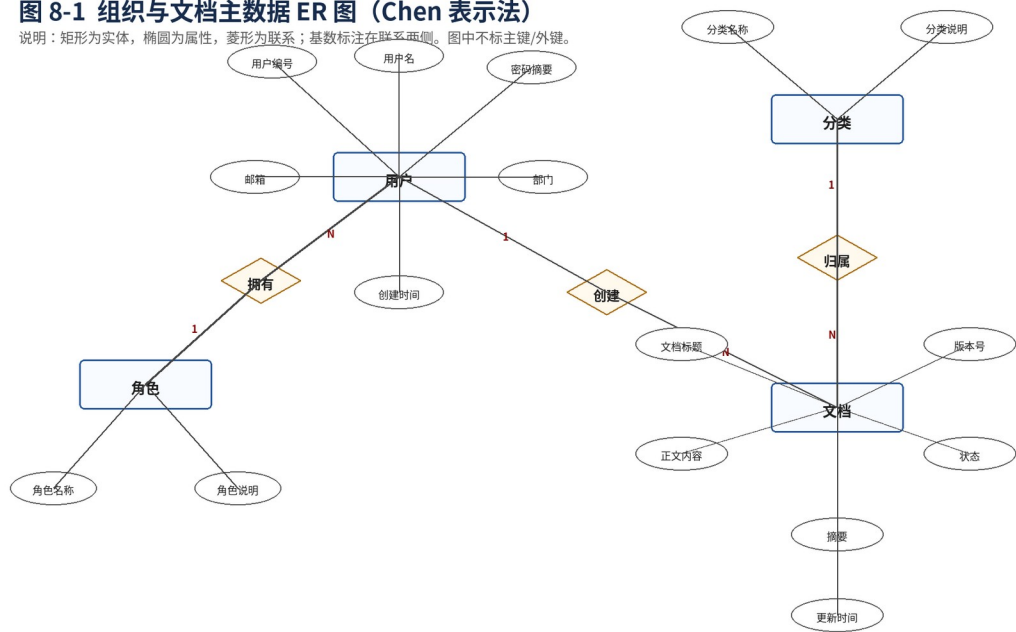


图 8-1 说明：本图体现平台最核心的主数据结构。用户通过“拥有”联系归属于角色；用户通过“创建”联系维护文档；文档通过“归属”联系落入某一分类。

8.2 图 8-2 文档扩展信息与用户行为

图 8-2 文档扩展信息与用户行为 ER 图（Chen 表示法）

说明：为保证可读性，本图聚焦“标签、版本、阅读、收藏”四类业务。

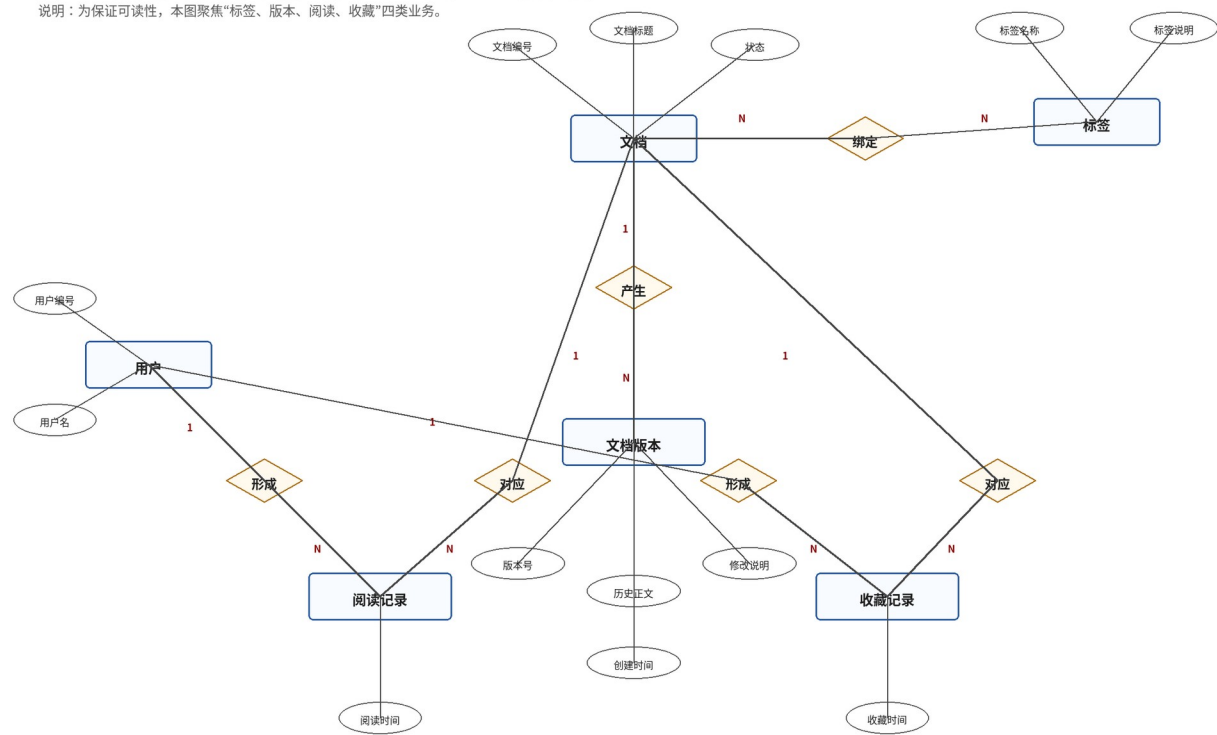


图 8-2 说明：本图体现文档的扩展属性和用户行为数据。文档可绑定多个标签、产生多个版本；用户产生阅读记录和收藏记录，而每条记录又对应某一篇文档。

8.3 图 8-3 智能问答与 Agent 处理

图 8-3 智能问答与 Agent 处理 ER 图 (Chen 表示法)

说明：本图聚焦“提问-回答-引用文档-智能处理-FAQ”链路。

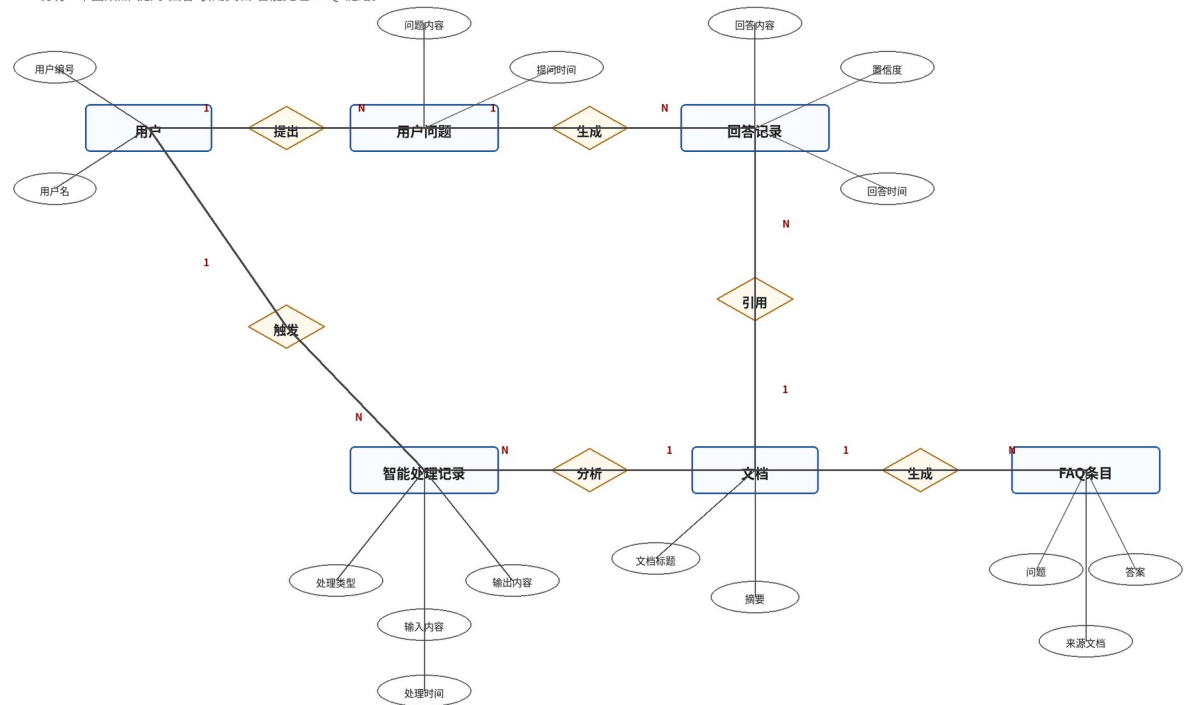


图 8-3 说明：本图体现用户提问、系统回答、文档引用、FAQ 生成和 Agent 处理留痕的完整链路。用户提出问题后生成回答记录，回答记录引用文档；文档还能生成 FAQ 条目，同时用户可触发文档的智能处理。

9. 联系说明表

联系名称	参与实体	基数	业务含义
拥有	角色 — 用户	1 : N	一个角色下可包含多个用户，一个用户只隶属于一个角色。
创建	用户 — 文档	1 : N	一个用户可以创建多篇文档，一篇文档只有一个创建者。
归属	分类 — 文档	1 : N	一个分类下有多篇文档，一篇文档归属于一个分类。
绑定	文档 — 标签	N : N	一篇文档可拥有多个标签，一个标签可被多篇文档复用。
产生	文档 — 文档版本	1 : N	一篇文档可派生多个历史版本。
形成	用户 — 阅读记录	1 : N	一个用户可形成多条阅读记录。
对应	文档 — 阅读记录	1 : N	一篇文档会对应多条阅读记录。
形成	用户 — 收藏记录	1 : N	一个用户可形成多条收藏记录。
对应	文档 — 收藏记录	1 : N	一篇文档会对应多条收藏记录。
提出	用户 — 用户问题	1 : N	一个用户可提出多个问题。
生成	用户问题 — 回答记录	1 : N	一个问题可对应一条或多条回答记录。

引用	回答记录 — 文档	N : 1	多条回答记录可以引用同一文档。
生成	文档 — FAQ 条目	1 : N	一篇文档可生成多个 FAQ 条目。
触发	用户 — 智能处理记录	1 : N	一个用户可触发多次智能处理。
分析	文档 — 智能处理记录	1 : N	一篇文档可产生多条智能处理记录。

10. 关系模型汇总表与详细表

说明：以下关系模型表采用“逻辑中文名 + 物理表名 + 字段级键说明”的呈现方式。主键标注为 PK，外键标注为 FK→目标表。

关系模型（中文）	物理表名	字段构成与键
角色	roles	角色编号(PK), 角色名称, 角色说明
用户	users	用户编号(PK), 角色编号(FK→roles), 用户名, 密码摘要, 邮箱, 部门, 创建时间
分类	categories	分类编号(PK), 分类名称, 分类说明
文档	documents	文档编号(PK), 分类编号(FK→categories), 创建者编号(FK→users), 文档标题, 正文内容, 摘要, 状态, 版本号, 创建时间, 更新时间
标签	tags	标签编号(PK), 标签名称, 标签说明
文档标签关联	document_tags	关联编号(PK), 文档编号(FK→documents), 标签编号(FK→tags)
文档版本	document_versions	版本记录编号(PK), 文档编号(FK→documents), 版本号, 历史正文, 修改说明, 创建时间
阅读记录	read_records	阅读记录编号(PK), 用户编号(FK→users), 文档编号(FK→documents), 阅读时间
收藏记录	favorite_records	收藏记录编号(PK), 用户编号(FK→users), 文档编号(FK→documents), 收藏时间
FAQ 条目	faq_items	FAQ 编号(PK), 文档编号(FK→documents), 问题, 答案, 创建时间
用户问题	questions	问题编号(PK), 用户编号(FK→users), 问题内容, 提问时间
回答记录	answers	回答编号(PK), 问题编号(FK→questions), 引用文档编号(FK→documents), 回答内容, 置信度, 回答时间
智能处理记录	agent_records	处理记录编号(PK), 文档编号(FK→documents), 触发用户编号(FK→users), 处理类型, 输入内容, 输出内容, 处理时间

角色（roles）

中文字段名	物理字段名	类型	键类型	说明
角色编号	role_id	BIGINT	PK	自增主键
角色名称	role_name	VARCHAR(50)		如管理员、知识管理员、普通员工
角色说明	description	VARCHAR(255)		角色说明

用户（users）

中文字段名	物理字段名	类型	键类型	说明
用户编号	user_id	BIGINT	PK	自增主键
角色编号	role_id	BIGINT	FK→roles.role_id	用户所属角色
用户名	username	VARCHAR(64)	UNIQUE	登录名
密码摘要	password_hash	VARCHAR(255)		密码加密摘要
邮箱	email	VARCHAR(128)		联系邮箱
部门	department	VARCHAR(128)		所属部门
创建时间	created_at	DATETIME		创建时间

分类（categories）

中文字段名	物理字段名	类型	键类型	说明
分类编号	category_id	BIGINT	PK	自增主键
分类名称	category_name	VARCHAR(100)	UNIQUE	如公司制度、技术文档
分类说明	description	VARCHAR(255)		分类描述

文档（documents）

中文字段名	物理字段名	类型	键类型	说明
文档编号	document_id	BIGINT	PK	自增主键
分类编号	category_id	BIGINT	FK→categories.category_id	所属分类
创建者编号	creator_id	BIGINT	FK→users.user_id	创建文档的用户
文档标题	title	VARCHAR(200)		标题
正文内容	content	LONGTEXT		文档正文
摘要	summary	TEXT		人工或 Agent 生成摘要
状态	status	VARCHAR(20)		草稿/已发布/已归档
版本号	version_no	VARCHAR(20)		当前版本号
创建时间	created_at	DATETIME		创建时间
更新时间	updated_at	DATETIME		最近更新时间

标签（tags）

中文字段名	物理字段名	类型	键类型	说明
标签编号	tag_id	BIGINT	PK	自增主键
标签名称	tag_name	VARCHAR(50)	UNIQUE	标签文本
标签说明	description	VARCHAR(255)		标签说明

文档标签关联（document_tags）

中文字段名	物理字段名	类型	键类型	说明
关联编号	id	BIGINT	PK	自增主键
文档编号	document_id	BIGINT	FK→documents.document_id	关联文档
标签编号	tag_id	BIGINT	FK→tags.tag_id	关联标签

文档版本（document_versions）

中文字段名	物理字段名	类型	键类型	说明
版本记录编号	version_id	BIGINT	PK	自增主键

文档编号	document_id	BIGINT	FK→documents.document_id	所属文档
版本号	version_no	VARCHAR(20)		历史版本号
历史正文	content	LONGTEXT		该版本正文
修改说明	change_note	VARCHAR(255)		变更说明
创建时间	created_at	DATETIME		版本创建时间

阅读记录 (read_records)

中文字段名	物理字段名	类型	键类型	说明
阅读记录编号	read_id	BIGINT	PK	自增主键
用户编号	user_id	BIGINT	FK→users.user_id	阅读者
文档编号	document_id	BIGINT	FK→documents.document_id	被阅读文档
阅读时间	read_time	DATETIME		阅读时间

收藏记录 (favorite_records)

中文字段名	物理字段名	类型	键类型	说明
收藏记录编号	favorite_id	BIGINT	PK	自增主键
用户编号	user_id	BIGINT	FK→users.user_id	收藏者
文档编号	document_id	BIGINT	FK→documents.document_id	被收藏文档
收藏时间	favorite_time	DATETIME		收藏时间

FAQ 条目 (faq_items)

中文字段名	物理字段名	类型	键类型	说明
FAQ 编号	faq_id	BIGINT	PK	自增主键
文档编号	document_id	BIGINT	FK→documents.document_id	来源文档
问题	question	VARCHAR(255)		FAQ 问题
答案	answer	TEXT		FAQ 答案
创建时间	created_at	DATETIME		创建时间

用户问题 (questions)

中文字段名	物理字段名	类型	键类型	说明
问题编号	question_id	BIGINT	PK	自增主键
用户编号	user_id	BIGINT	FK→users.user_id	提问用户
问题内容	question_text	TEXT		用户问题文本
提问时间	created_at	DATETIME		提问时间

回答记录 (answers)

中文字段名	物理字段名	类型	键类型	说明
回答编号	answer_id	BIGINT	PK	自增主键
问题编号	question_id	BIGINT	FK→questions.question_id	来源问题
引用文档编号	document_id	BIGINT	FK→documents.docum	主要引用文档

			ent_id	
回答内容	answer_text	TEXT		系统答案
置信度	confidence	DECIMAL(5,2)		可选评分
回答时间	created_at	DATETIME		回答时间

智能处理记录 (agent_records)

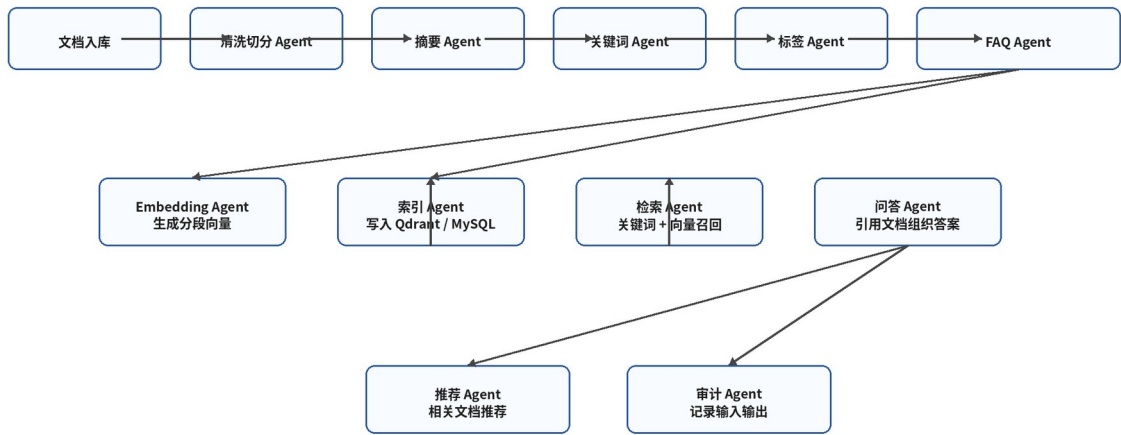
中文字段名	物理字段名	类型	键类型	说明
处理记录编号	record_id	BIGINT	PK	自增主键
文档编号	document_id	BIGINT	FK→documents.document_id	被处理文档
触发用户编号	user_id	BIGINT	FK→users.user_id	触发处理的用户
处理类型	agent_type	VARCHAR(50)		摘要生成/标签推荐/FAQ生成/问答
输入内容	input_text	LONGTEXT		处理输入
输出内容	output_text	LONGTEXT		处理输出
处理时间	created_at	DATETIME		记录时间

11. Agent 体系与 RAG 流程

本系统不把“Agent”作为单一聊天窗口，而是设计为一组围绕知识生命周期工作的智能体：清洗切分 Agent、摘要 Agent、关键词 Agent、标签 Agent、FAQ Agent、Embedding Agent、检索 Agent、问答 Agent、推荐 Agent、审计 Agent。

图 11-1 多 Agent 协同 workflow

课程阶段允许先用规则模板模拟 Agent 输出，接口层保持兼容 OpenAI 协议，后续可平滑切换真实模型。



11.1 各 Agent 职责

Agent 名称	输入	输出	作用
清洗切分 Agent	文档标题、正文	标准化文本、分段片段	为 Embedding 和检索做预处理。
摘要 Agent	文档正文	摘要文本	用于列表页摘要展示和搜索结果概览。
关键词 Agent	文档正文	关键词数组	为标签和检索增强服务。
标签 Agent	标题、摘要、关键词	推荐标签	降低人工贴标签成本。
FAQ Agent	文档正文	FAQ 条目列表	沉淀常见问答。
Embedding Agent	分段文本	向量数组	写入 Qdrant 建立向量索引。
检索 Agent	用户问题	候选文档片段	联合关键词检索与向量召回。
问答 Agent	用户问题、候选片段	结构化答案	面向用户输出可读答案。
推荐 Agent	当前文档、标签、向量	相关文档列表	做相关推荐。
审计 Agent	所有 Agent 输入输出	审计记录	为追踪和测试提供留痕。

11.2 RAG 执行策略

推荐采用“结构化字段检索 + 向量召回 + 大模型组织答案”的三级流程：

- 步骤 1：先根据标题、摘要、标签、分类做 MySQL 关键词初筛。
- 步骤 2：调用 Embedding 接口生成问题向量，在 Qdrant 中召回相似片段。
- 步骤 3：合并候选片段，交给问答 Agent 生成答案，并返回引用文档。
- 步骤 4：记录问题、答案、命中的文档与执行时间。

12. 前端详细设计（TypeScript）

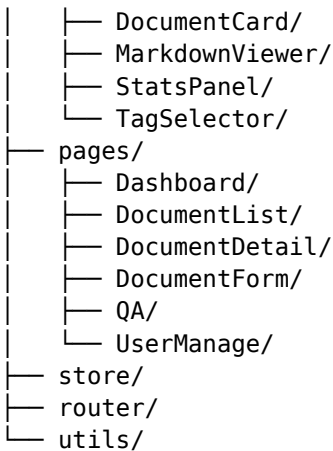
前端采用 React 18 + TypeScript + Vite + Ant Design。状态管理推荐 Zustand（轻量）或 Redux Toolkit（规范）。图表使用 ECharts，网络请求使用 Axios。

12.1 页面结构

页面路由	页面名称	核心功能
/login	登录页	用户名密码登录、表单校验
/dashboard	首页看板	文档统计、热门文档、问答趋势、最近更新
/documents	文档列表	搜索、筛选、分页、批量操作
/documents/:id	文档详情	正文、摘要、标签、版本、相关推荐
/documents/new	文档新增/编辑	录入文档并触发 Agent
/categories	分类管理	分类 CRUD
/tags	标签管理	标签 CRUD
/qa	智能问答	提问、展示答案、引用文档
/agent-records	智能处理记录	查看摘要/标签/FAQ/问答日志
/users	用户管理	用户与角色维护

12.2 前端组件分层

```
src/
├── api/                # request.ts, auth.ts, document.ts, qa.ts
├── components/
│   ├── Layout/
│   └── SearchBar/
```



12.3 前端交互要求

- (1) 文档列表支持标题关键词、分类、标签、多条件筛选。
- (2) 文档详情页需展示摘要、正文、标签、版本、相关推荐、收藏按钮。
- (3) 问答页需展示问题、答案、引用文档、置信度与历史提问。
- (4) 所有表单须做前端校验，并对异步请求展示加载态、成功态和失败态。

13. 后端详细设计（Rust）

后端建议采用 Rust 1.78+、Axum、Tokio、SQLx、Serde、Tracing。Axum 负责路由与中间件，Service 层承载业务逻辑，Repository 层承接 MySQL/Qdrant/Redis 访问。

13.1 后端分层

层次	职责
API 层	接收 HTTP 请求、参数校验、返回统一响应。
Service 层	编排业务流程，例如新增文档后同时调用摘要 Agent 和标签 Agent。
Repository 层	封装对 MySQL、Qdrant、Redis 的访问。
Agent 层	封装与大模型 API 的交互与 Prompt 模板。
RAG 层	实现文档切分、Embedding、召回、重排。
Middleware 层	JWT 鉴权、日志记录、异常处理、跨域配置。

13.2 推荐 Crates

用途	推荐库
Web 框架	axum
异步运行时	tokio
MySQL 访问	sqlx
序列化	serde / serde_json
配置管理	config / envy
日志	tracing / tracing-subscriber
JWT	jsonwebtoken
HTTP 客户端	reqwest

时间处理	chrono
唯一 ID	uuid
测试	tokio-test / rstest

13.3 Rust 模块建议

```
src/
├── main.rs
├── app/mod.rs
├── api/
│   ├── auth.rs
│   ├── document.rs
│   ├── category.rs
│   ├── tag.rs
│   ├── qa.rs
│   └── agent.rs
├── service/
│   ├── auth_service.rs
│   ├── document_service.rs
│   ├── qa_service.rs
│   └── agent_service.rs
├── repository/
│   ├── mysql/
│   ├── qdrant/
│   └── redis/
├── agent/
│   ├── client.rs
│   ├── prompts.rs
│   ├── summarizer.rs
│   ├── tagger.rs
│   ├── faq.rs
│   └── qa_agent.rs
└── rag/
    ├── splitter.rs
    ├── embedding.rs
    ├── retriever.rs
    └── reranker.rs
```

14. 大模型、向量库与检索配置

14.1 推荐默认方案

组件	推荐方案	原因
聊天模型	兼容 OpenAI 协议的模型，例如 deepseek-chat / qwen-max / gpt-4o-mini	Rust 侧统一使用 request 调 OpenAI-Compatible API，切换成本低。
Embedding 模型	text-embedding-v3 或 BGE-M3 服务化接口	便于将文本片段写入 Qdrant。
向量数据库	Qdrant	开源、部署简单、与 RAG 场景匹配。
结构化数据库	MySQL 8.0	满足课程数据库设计与事务需求。
缓存	Redis 7	缓存会话、热点检索与限流信息。

14.2 文档切分建议

- (1) 制度类、技术类和项目类文档，建议按“标题 + 段落”切分，分段长度 300~600 中文字符。
- (2) FAQ 文档按单问单答切分。
- (3) 每个分段保存：segment_id、document_id、chunk_text、embedding、chunk_order、tokens。

14.3 问答 Prompt 建议

是企知答助手。依据定考文回答。

要求：

1. 若参考文档中没有足够依据，请明确说明“暂未找到明确依据”；
2. 回答后给出引用文档标题；
3. 用条理清晰的中文作答。

用户问题：{question}

参考片段：

{context}

15. 环境变量与配置文件架构

15.1 根目录配置策略

建议采用“前后端分层 + 环境区分”的配置策略：frontend/.env.development、frontend/.env.production、backend/.env、infra/.env。所有敏感信息只进入后端 .env，不进入前端仓库。

15.2 后端 .env.example

```
APP_NAME=zhishu-backend
APP_ENV=development
APP_HOST=0.0.0.0
APP_PORT=8080
APP_BASE_URL=http://localhost:8080
JWT_SECRET=replace_me_with_strong_secret
JWT_EXPIRE_HOURS=72

MYSQL_URL=mysql://root:password@127.0.0.1:3306/zhishu
MYSQL_MAX_CONNECTIONS=20
MYSQL_MIN_CONNECTIONS=5

REDIS_URL=redis://127.0.0.1:6379
REDIS_PREFIX=zhishu:

QDRANT_URL=http://127.0.0.1:6333
QDRANT_API_KEY=
QDRANT_COLLECTION=zhishu_docs

MINIO_ENDPOINT=http://127.0.0.1:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET=zhishu

LLM_BASE_URL=https://api.openai.com/v1
LLM_API_KEY=your_api_key
```



```
LLM_CHAT_MODEL=deepseek-chat
LLM_EMBED_MODEL=text-embedding-3-small
LLM_TIMEOUT_SECONDS=60

RAG_TOP_K=5
RAG_SCORE_THRESHOLD=0.45
RAG_CHUNK_SIZE=500
RAG_CHUNK_OVERLAP=80

LOG_LEVEL=info
CORS_ALLOW_ORIGIN=http://localhost:5173
```

15.3 前端 .env.development.example

```
VITE_APP_TITLE=知枢
VITE_API_BASE_URL=http://localhost:8080/api
VITE_ENABLE MOCK=false
VITE_ROUTER_MODE=history
```

15.4 环境变量说明表

变量名	所属层	说明
MYSQL_URL	后端	MySQL 连接串
QDRANT_URL	后端	向量数据库地址
REDIS_URL	后端	缓存服务地址
LLM_BASE_URL	后端	大模型网关地址
LLM_API_KEY	后端	大模型密钥
LLM_CHAT_MODEL	后端	聊天模型标识
LLM_EMBED_MODEL	后端	Embedding 模型标识
RAG_TOP_K	后端	检索召回条数
VITE_API_BASE_URL	前端	后端 API 基地址

16. 接口设计与交互流程

接口路径	方法	功能说明
/api/auth/login	POST	用户登录，返回 JWT。
/api/users	GET	管理员获取用户列表。
/api/documents	GET	查询文档列表。
/api/documents	POST	新增文档。
/api/documents/{id}	PUT	修改文档。
/api/documents/{id}	DELETE	删除文档。
/api/categories	GET/POST/PUT/DELETE	分类管理。
/api/tags	GET/POST/PUT/DELETE	标签管理。
/api/documents/{id}/summary	POST	触发摘要 Agent。
/api/documents/{id}/faq	POST	触发 FAQ Agent。
/api/qa/ask	POST	提交问题并返回答案。
/api/agent-records	GET	查看智能处理记录。

16.1 文档入库流程

前端提交新增文档 -> Rust 后端写入 documents -> 调用摘要 Agent 与标签 Agent -> 写入 summary 与 document_tags -> 切分正文并调用 Embedding -> 向量写入 Qdrant -> 记录 agent_records。

16.2 问答流程

前端提交问题 -> 后端在 MySQL 中做关键词初筛 -> 生成问题向量到 Qdrant 召回 -> 组织 Prompt 调用问答 Agent -> 写入 questions 与 answers -> 返回答案、引用文档和置信度。

17. 开发执行计划

阶段	周期	主要任务	交付物
阶段一	第 1 周	确定选题、角色、功能范围、模拟资料目录	需求说明、资料清单
阶段二	第 2 周	绘制 ER 图、完成关系模型、初始化 MySQL	ER 图、关系模型表、SQL 脚本
阶段三	第 3 周	完成前端静态页面与路由骨架	前端页面原型
阶段四	第 4 周	完成 Rust 后端基础 API 与登录鉴权	后端脚手架与基础接口
阶段五	第 5 周	实现文档 CRUD、分类标签、版本、阅读收藏	核心业务可跑通
阶段六	第 6 周	实现摘要/标签/FAQ/问答 Agent 与 Qdrant 检索	RAG 与多 Agent 模块
阶段七	第 7 周	完成统计看板、测试、答辩材料	测试报告、PPT、演示视频/脚本

18. 测试计划

编号	测试模块	测试内容	预期结果
TC01	登录	正确账号密码登录	返回 JWT 并进入首页
TC02	文档管理	新增一篇技术文档	列表出现新文档
TC03	文档管理	修改文档后保存	详情页显示更新内容
TC04	标签管理	为文档绑定标签	文档详情显示标签
TC05	版本管理	更新正文并写入版本说明	版本表新增一条记录
TC06	阅读记录	打开文档详情	写入阅读记录
TC07	收藏记录	点击收藏文档	写入收藏记录
TC08	摘要 Agent	触发摘要生成	summary 字段写入内容
TC09	FAQ Agent	触发 FAQ 生成	faq_items 新增多条记录
TC10	问答流程	提问“如何申请数据库权限”	返回答案并显示引用文档
TC11	检索流程	关键词搜索“报销”	返回报销相关文档
TC12	看板	进入首页看板	显示总量、趋势、热门文档

19. 部署方案与运维建议

开发环境：Windows 或 Linux。前端使用 Node.js 20+，后端使用 Rust stable，MySQL 8.0、Qdrant 1.9+、Redis 7。

推荐用 Docker Compose 编排 MySQL、Qdrant、Redis、MinIO，前端和后端在本地启动。若需要统一部署，可使用 Nginx 反向代理 frontend 构建产物与 backend API。

```
services:
  mysql:
```

```
redis:
qdrant:
minio:
backend:
frontend:
```

运维建议：

(1) LLM 调用设置超时与重试；(2) Agent 输出统一记录到 agent_records；(3) Qdrant 建索引前先去重；(4) 用户上传文档内容要做长度限制；(5) 生产环境关闭调试日志。

20. 答辩展示建议

展示顺序建议如下：

1) 简述业务痛点与系统目标；2) 展示三张拆分后的中文 ER 图；3) 说明关系模型表如何标注 PK/FK；4) 演示前端首页、文档列表、文档详情；5) 新增一篇文档并触发摘要 Agent；6) 展示问答页提问并引用文档；7) 查看智能处理记录与统计看板；8) 总结多 Agent、RAG、Rust 后端、TypeScript 前端等技术亮点。

答辩时可强调：本系统并非“只有 AI 功能”，而是一个完整的企业知识管理 Web 系统，AI 只是嵌入其中增强检索、摘要、问答和推荐能力。